

Skill-Week4

Session-7:

1. Process Synchronization and Child Process Monitoring Using waitpid()

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("Child process started\n");
        printf("Child PID: %d\n", getpid());

        printf("Child is working...\n");
        sleep(5);

        printf("Child process completed.\n");

        exit(0);
    }
    else {
        // Parent process
        printf("Parent process started\n");
        printf("Parent PID: %d\n", getpid());
        printf("Waiting for child PID: %d\n", pid);
    }
}
```

```

pawan_kumar@LAPTOP-SSD0PUQ2:~$ nano waitpid_demo.c
pawan_kumar@LAPTOP-SSD0PUQ2:~$ gcc waitpid_demo.c -o waitpid_demo
pawan_kumar@LAPTOP-SSD0PUQ2:~$ ./waitpid_demo
Parent process started
Parent PID: 759
Waiting for child PID: 760
Child process started
Child PID: 760
Child is working...
Child process completed.
Child process has been collected.
Child exited normally.
Exit status: 0
Parent process completed.
pawan_kumar@LAPTOP-SSD0PUQ2:~$ |

```

Report: The waitpid() function is used for process synchronization and monitoring a specific child process. The parent process waits until the specified child completes and then collects its exit status. This ensures proper synchronization and prevents zombie processes.

2. To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

int main() {
    char command[100];
    char *path;
    char *pathCopy;
    char *directory;
    char fullPath[PATH_MAX];

    // Retrieve PATH variable
    path = getenv("PATH");

    if (path == NULL) {
        printf("PATH variable not found.\n");
        return 1;
    }

    printf("PATH Variable:\n%s\n\n", path);

    // Get command from user
    printf("Enter a command: ");
    scanf("%99s", command);

    // Make a copy of PATH because strtok modifies the string
    pathCopy = strdup(path);

    if (pathCopy == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }
}

```

```

pawan_kumar@LAPTOP-SSD0PUQ2:~$ nano path_search.c
pawan_kumar@LAPTOP-SSD0PUQ2:~$ gcc path_search.c -o path_search
pawan_kumar@LAPTOP-SSD0PUQ2:~$ ./path_search
PATH Variable:
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr
/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/WindowsApps/MicrosoftCorp
orationII.WindowsSubsystemForLinux_2.7.11.0_x64__8wekyb3d8bbwe:/mnt/c/Progra
m Files (x86)/Common Files/Oracle/Java/java8path:/mnt/c/Program Files (x86)/
Common Files/Oracle/Java/javapath:/mnt/c/Windows/system32:/mnt/c/Windows:/mn
t/c/Windows/System32/Wbem:/mnt/c/Windows/System32/WindowsPowerShell/v1.0:/m
nt/c/Windows/System32/OpenSSH:/mnt/c/Program Files (x86)/NVIDIA Corporation
/PhysX/Common:/mnt/c/Program Files/dotnet:/mnt/c/Program Files/Java/jdk-21/
bin:/mnt/c/Program Files/NVIDIA Corporation/NVIDIA App/NvDLISR:/mnt/c/Progra
m Files/nodejs:/mnt/c/Program Files/Git/cmd:/mnt/c/Program Files/MySQL/MySQ
L Shell 8.0/bin:/mnt/c/Users/PAWAN KUMAR/.local/bin:/mnt/c/Users/PAWAN KUMA
R/AppData/Local/Blackbox/bin:/mnt/c/Users/PAWAN KUMAR/AppData/Local/Microsof
t/WindowsApps:/mnt/c/Users/PAWAN KUMAR/AppData/Local/Programs/Microsoft VS C
ode/bin:/mnt/c/Users/PAWAN KUMAR/AppData/Roaming/npm:/mnt/c/Users/PAWAN KUMA
R/AppData/Local/Python/bin:/snap/bin

Enter a command: ls

Command found: ls
Executable location: /usr/bin/ls
pawan_kumar@LAPTOP-SSD0PUQ2:~$ |

```

```

pawan_kumar@LAPTOP-SSD0PUQ2:~$ ./path_search
PATH Variable:
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr
/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/WindowsApps/MicrosoftCorp
orationII.WindowsSubsystemForLinux_2.7.11.0_x64__8wekyb3d8bbwe:/mnt/c/Progra
m Files (x86)/Common Files/Oracle/Java/java8path:/mnt/c/Program Files (x86)/
Common Files/Oracle/Java/javapath:/mnt/c/Windows/system32:/mnt/c/Windows:/mn
t/c/Windows/System32/Wbem:/mnt/c/Windows/System32/WindowsPowerShell/v1.0:/m
nt/c/Windows/System32/OpenSSH:/mnt/c/Program Files (x86)/NVIDIA Corporation
/PhysX/Common:/mnt/c/Program Files/dotnet:/mnt/c/Program Files/Java/jdk-21/
bin:/mnt/c/Program Files/NVIDIA Corporation/NVIDIA App/NvDLISR:/mnt/c/Progra
m Files/nodejs:/mnt/c/Program Files/Git/cmd:/mnt/c/Program Files/MySQL/MySQ
L Shell 8.0/bin:/mnt/c/Users/PAWAN KUMAR/.local/bin:/mnt/c/Users/PAWAN KUMA
R/AppData/Local/Blackbox/bin:/mnt/c/Users/PAWAN KUMAR/AppData/Local/Microsof
t/WindowsApps:/mnt/c/Users/PAWAN KUMAR/AppData/Local/Programs/Microsoft VS C
ode/bin:/mnt/c/Users/PAWAN KUMAR/AppData/Roaming/npm:/mnt/c/Users/PAWAN KUMA
R/AppData/Local/Python/bin:/snap/bin

Enter a command: ls

Command found: ls
Executable location: /usr/bin/ls
pawan_kumar@LAPTOP-SSD0PUQ2:~$ |

```

Report: The program successfully retrieved the PATH environment variable and parsed its directories to search for a specified command. It located executable files and verified their execution permissions using the access() function. It also handled missing commands correctly by displaying an appropriate error message. Thus, the program

demonstrated how the operating system resolves commands using the PATH variable.

Session-8:

- 1. To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic**

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 100
void expandVariable(char input[], char output[]) {
    int i = 0;
    int j = 0;
    while (input[i] != '\0') {
        if (input[i] == '$') {
            i++;
            char variable[MAX];
            int k = 0;
            while (input[i] != '\0' &&
                    input[i] != ' ' &&
                    input[i] != '$') {
                variable[k] = input[i];
                k++;
                i++;
            }
            variable[k] = '\0';
            char *value = getenv(variable);
            if (value != NULL) {
                strcat(output, value);
            }
            else {
                printf("Warning: Variable %s is undefined\n", variable);
            }
        }
        else {
            int len = strlen(output);
            output[len] = input[i];
            output[len + 1] = '\0';
            i++;
        }
    }
}

```

```

pawan_kumar@LAPTOP-SSD0PUQ2:~$ nano variable_expand.c
pawan_kumar@LAPTOP-SSD0PUQ2:~$ gcc variable_expand.c -o variable_expand
pawan_kumar@LAPTOP-SSD0PUQ2:~$ ./variable_expand
Enter a command or text: $HOME

Original Input: $HOME
Expanded Output: /home/pawan_kumar
pawan_kumar@LAPTOP-SSD0PUQ2:~$ |

```

```

pawan_kumar@LAPTOP-SSD0PUQ2:~$ ./variable_expand
Enter a command or text: $HOME

Original Input: $HOME
Expanded Output: /home/pawan_kumar
pawan_kumar@LAPTOP-SSD0PUQ2:~$ |

```

Report: The program successfully detects variable references using the \$ symbol and expands them by retrieving their values from environment variables. It also handles

undefined variables by displaying an appropriate message. Thus, the program demonstrates the basic process of variable expansion and token updating used in command interpreters and shell programs.

2. To Identify Built-ins, Create Dispatch Table, Execute In Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>
void execute(char command[]) {
    if (strcmp(command, "pwd") == 0) {
        char path[100];
        getcwd(path, sizeof(path));
        printf("Current directory: %s\n", path);
    }
    else if (strncmp(command, "cd ", 3) == 0) {
        char *path = command + 3;
        if (chdir(path) == 0)
            printf("Directory changed\n");
        else
            printf("Invalid directory\n");
    }
    else if (strcmp(command, "exit") == 0) {
        printf("Exiting...\n");
        exit(0);
    }
    else {
        printf("Invalid command\n");
    }
}
int main() {
    char command[100];
    while (1) {
        printf("MyShell> ");
        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';
        execute(command);
    }
    return 0;
}
```

```
Expanded output: /home/pawan_kumar  
pawan_kumar@LAPTOP-SSD0PUQ2:~$ nano builtin.c  
pawan_kumar@LAPTOP-SSD0PUQ2:~$ gcc builtin.c -o builtin  
pawan_kumar@LAPTOP-SSD0PUQ2:~$ ./builtin  
MyShell> pwd  
Current directory: /home/pawan_kumar  
MyShell> cd /tmp  
Directory changed  
MyShell> pwd  
Current directory: /tmp  
MyShell> hello  
Invalid command  
MyShell> exit  
Exiting...  
pawan_kumar@LAPTOP-SSD0PUQ2:~$ |
```

Report: The program identifies built-in commands and executes them within the same process. It handles commands using conditional dispatch, maintains the current directory state using `cd`, and displays an error message for invalid commands.